第10章　条条大路通罗马——单表访问方法

对于 MySQL 用户来说，MySQL 其实就是一个软件，平时用的最多的就是查询功能。DBA 时不时丢过来一些慢查询语句让我们优化，如果我们连查询是怎么执行的都不清楚，优化也就无从谈起了。所以，是时候掌握真正的技术了。

第 1 章曾经讲过，MySQL Server 有一个称为优化器的模块。MySQL Server 在对一条查询语句进行语法解析之后，就会将其交给优化器来优化，优化的结果就是生成一个所谓的执行计划。这个执行计划表明了应该使用哪些索引进行查询、表之间的连接顺序是啥样的；等等。最后会按照执行计划中的步骤调用存储引擎提供的接口来真正地执行查询，并将查询结果返给客户端。

不过查询优化这个主题有点儿大，在学会跑之前得先学会走，所以本章先来瞅瞅 MySQL 怎么执行单表查询的（就是 FROM 子句后面只有一个表）。需要强调的一点是，在学习本章前大家一定要先仔细看过前面章节中关于记录结构、数据页结构以及索引的内容，并确保已经完全掌握了这些内容；反之，本章不合适你。

为了故事的顺利发展，我们还是得把老朋友 single_table 请出来。为了防止大家把这个老朋友忘掉了，我们再看一下它的表结构：

```
CREATE TABLE single_table (
    id INT NOT NULL AUTO_INCREMENT,
    key1 VARCHAR(100),
    key2 INT,
    key3 VARCHAR(100),
    key_part1 VARCHAR(100),
    key_part2 VARCHAR(100),
    key_part3 VARCHAR(100),
    common_field VARCHAR(100),
    PRIMARY KEY (id),
    KEY idx_key1 (key1),
    UNIQUE KEY uk_key2 (key2),
    KEY idx_key3 (key3),
    KEY idx_key_part(key_part1, key_part2, key_part3)
) Engine=InnoDB CHARSET=utf8;
```

我们为这个 single_table 表建立了 1 个聚簇索引和 4 个二级索引，分别是：

- 为 id 列建立的聚簇索引；
- 为 key1 列建立的 idx_key1 二级索引；
- 为 key2 列建立的 uk_key2 二级索引，而且该索引是唯一二级索引；
- 为 key3 列建立的 idx_key3 二级索引；
- 为 key_part1、key_part2、key_part3 列建立的 idx_key_part 二级索引，这也是一个联合索引。

接下来需要为这个表插入 10,000 行记录，除 id 列外，其余的列都插入随机值。具体的插入语句这里就不写了，大家自己写个程序插入吧（id 列是自增主键列，不需要手动插入）。

10.1 访问方法的概念

想必各位都用过各种地图 App 来查找到某个地方的路线吧。如果我们搜索从西安钟楼到大雁塔的路线，地图 App 会给出多种路线供我们选择。如果我们实在闲的没事儿干并且足够有钱，还可以用南辕北辙的方式绕地球一圈到达目的地。无论采用哪一种方式，我们最终的目标都是到达大雁塔这个地方。回到 MySQL 中来，我们平时所写的那些查询语句本质上只是一种声明式的语法，只是告诉 MySQL 要获取的数据符合哪些规则，至于 MySQL 背地里是如何把查询结果搞出来的则是 MySQL 自己的事儿。

设计 MySQL 的大叔把 MySQL 执行查询语句的方式称为访问方法（access method）或者访问类型。同一个查询语句可以使用多种不同的访问方法来执行，虽然最后的查询结果都是一样的，但是不同的执行方式花费的时间成本可能差距甚大。就像是从钟楼到大雁塔，你可以坐飞机去，坐公交车去，还可以骑共享单车去，当然也可以走着去。

下面将详细唠叨各种访问方法的具体内容。

10.2 const

有时可以通过主键列来定位一条记录，比如下面这个查询：

```
SELECT * FROM single_table WHERE id = 1438;
```

MySQL 会直接利用主键值在聚簇索引中定位对应的用户记录，如图 10-1 所示。

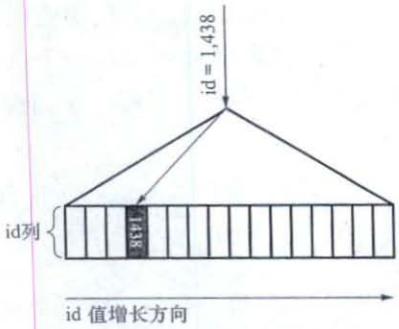

图 10-1　聚簇索引示意图

与之类似，我们根据唯一二级索引列来定位一条记录的速度也是贼快的。比如下面这个查询：

```
SELECT * FROM single_table WHERE key2 = 3841;
```

这个查询的执行过程的示意图如图 10-2 所示。

可以看到这个查询的执行分为下面两步。

步骤 1. 在 uk_key2 对应的 B+ 树索引中，根据 key2 列与常数的等值比较条件定位到一条二级索引记录。

步骤 2. 然后再根据该记录的 id 值到聚簇索引中获取到完整的用户记录。

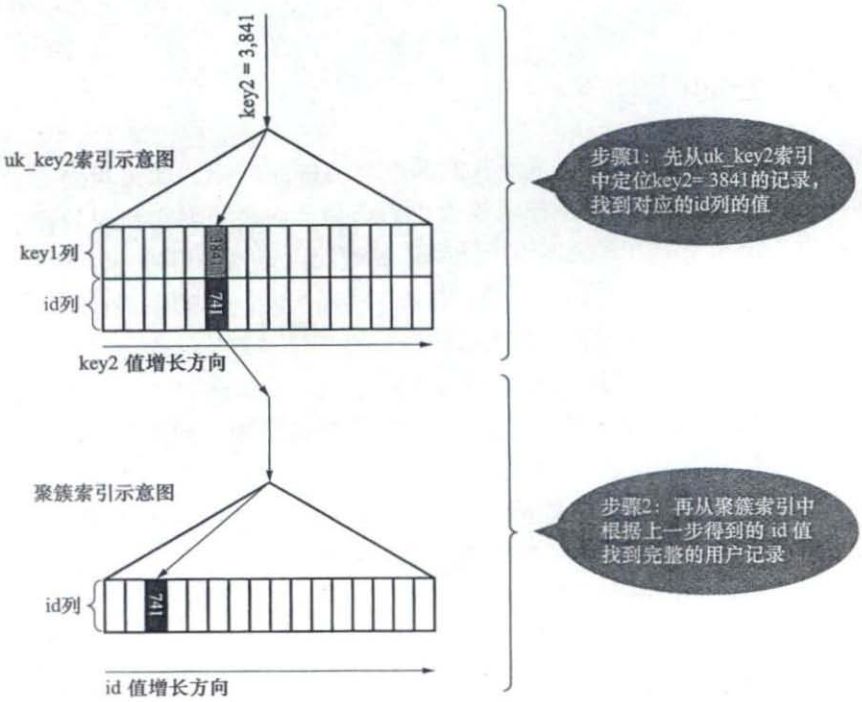

图 10-2　查询执行过程示意图

设计 MySQL 的大叔认为，通过主键或者唯一二级索引列与常数的等值比较来定位一条记录是像坐火箭一样快的，所以他们把这种通过主键或者唯一二级索引列来定位一条记录的访问方法定义为 const（意思是常数级别的，代价是可以忽略不计的）。不过这种 const 访问方法只能在主键列或者唯一二级索引列与一个常数进行等值比较时才有效。如果主键或者唯一二级索引的索引列由多个列构成，则只有在索引列中的每一个列都与常数进行等值比较时，这个 const 访问方法才有效（这是因为只有在该索引的每一个列都采用等值比较时，才可以保证最多只有一条记录符合搜索条件）。

对于唯一二级索引列来说，在查询列为 NULL 值时，情况比较特殊。比如下面这样：

```
SELECT * FROM single_table WHERE key2 IS NULL;
```

因为唯一二级索引列并不限制 NULL 值的数量，所以上述语句可能访问到多条记录。也就是说上面这个语句不可以使用 const 访问方法来执行（至于采用什么访问方法，会在下文介绍）。

10.3　ref

有时，我们需要将某个普通的二级索引列与常数进行等值比较，比如这样：

```
SELECT * FROM single_table WHERE key1 = 'abc';
```

对于这个查询，当然可以选择全表扫描的方式来执行。不过也可以使用 idx_key1 来执行，此时对应的扫描区间就是 ['abc', 'abc']，这也是一个单点扫描区间。我们可以定位到 key1 =

'abc' 条件的第一条记录，然后沿着记录所在的单向链表向后扫描，直到某条记录不符合 key1 = 'abc' 条件为止。由于查询列表是 *，因此针对获取到的每一条二级索引记录，都需要根据该记录的 id 值执行回表操作，到聚簇索引中获取到完整的用户记录后再发送给客户端。

由于普通二级索引并不限制索引列值的唯一性，所以位于扫描区间 ['abc', 'abc'] 中的二级索引记录可能有多条，此时使用二级索引执行查询的代价就取决于该扫描区间中的记录条数。如果该扫描区间中的记录较少，则回表操作的代价还是比较低的。设计 MySQL 的大叔把这种“搜索条件为二级索引列与常数进行等值比较，形成的扫描区间为单点扫描区间，采用二级索引来执行查询”的访问方法称为 ref。我们看一下如何采用 ref 访问方法执行查询，如图 10-3 所示。

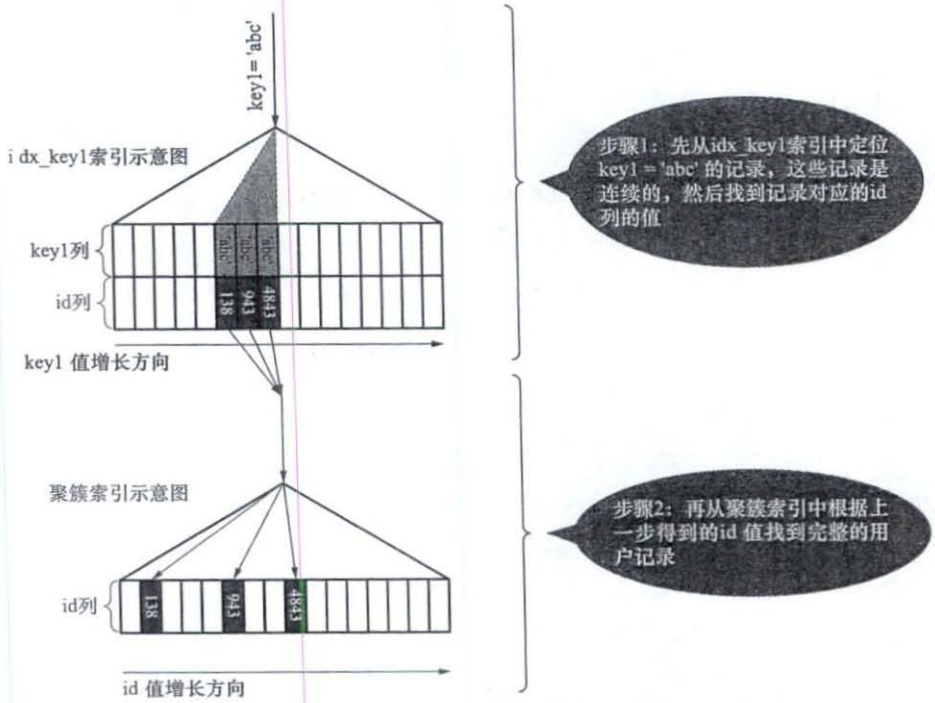

图 10-3 ref 访问方法执行查询示意图

小贴士

采用二级索引来执行查询时，其实每获取到一条二级索引记录，就会立刻对其执行回表操作，而不是将所有二级索引记录的主键值都收集起来后再统一执行回表操作。图 10-3 中的步骤 1 和步骤 2 只是为了让大家更直观地区分扫描二级索引记录和回表操作，大家清楚执行的过程就好了。

从图 10-3 可以看出，对于普通的二级索引来说，通过索引列进行等值比较后可能会匹配到多条连续的二级索引记录，而不是像主键或者唯一二级索引那样最多只能匹配一条记录。所以这种 ref 访问方法比 const 差了那么一点。另外，这里大家需要注意下面两种情况。

- 在二级索引列允许存储 NULL 值时，无论是普通的二级索引，还是唯一二级索引，它们的索引列并不限制 NULL 值的数量，所以在执行包含“key IS NULL”形式的搜索条件的查询时，最多只能使用 ref 访问方法，而不能使用 const 访问方法。
- 对于索引列中包含多个列的二级索引来说，只要最左边连续的列是与常数进行等值比

较，就可以采用 ref 访问方法。比如下面这几个查询都可以采用 ref 访问方法执行：

```
SELECT * FROM single_table WHERE key_part1 = 'god like';

SELECT * FROM single_table WHERE key_part1 = 'god like' AND key_part2 = 'legendary';

SELECT * FROM single_table WHERE key_part1 = 'god like' AND key_part2 = 'legendary' AND
key_part3 = 'penta kill';
```

如果索引列中最左边连续的列不全部是等值比较的话，它的访问方法就不能称为 ref 了。比如下面这条语句（其实该语句利用 idx_key_part 索引的访问方法就是后文要介绍的 range）：

```
SELECT * FROM single_table WHERE key_part1 = 'god like' AND key_part2 > 'legendary';
```

10.4　ref_or_null

有时，我们不仅想找出某个二级索引列的值等于某个常数的记录，而且还想把该列中值为 NULL 的记录也找出来。比如下面这个查询：

```
SELECT * FROM single_table WHERE key1 = 'abc' OR key1 IS NULL;
```

当使用二级索引而不是全表扫描的方式执行该查询时，对应的扫描区间就是 [NULL, NULL] 以及 ['abc', 'abc']，此时执行这种类型的查询所使用的访问方法就称为 ref_or_null。ref_or_null 访问方法的执行过程如图 10-4 所示。

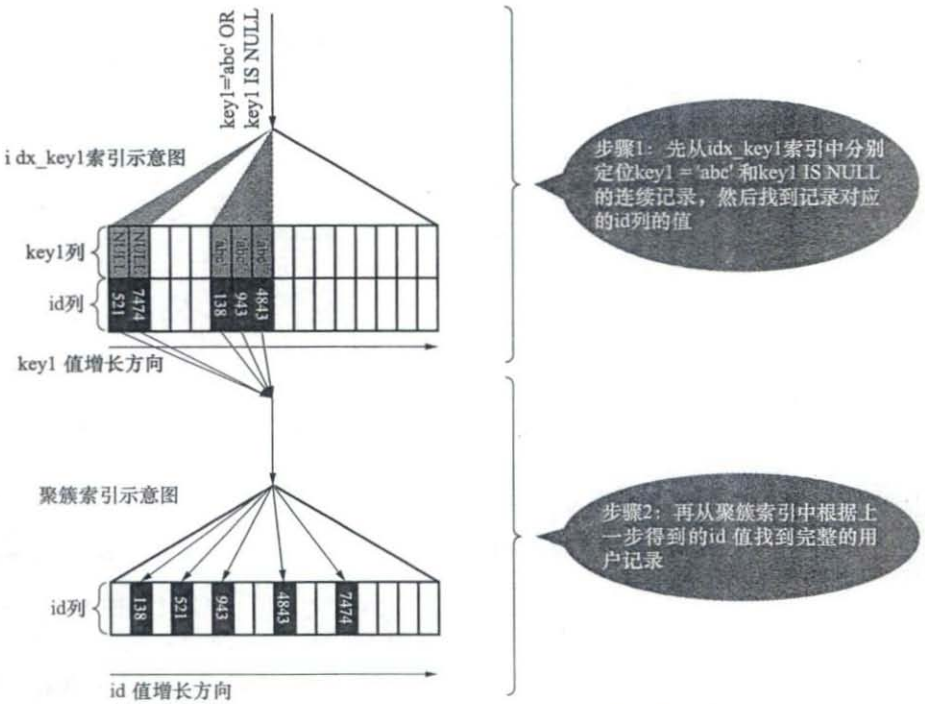

图 10-4　ref_or_null 访问方法的执行过程

可以看到，ref_or_null 访问方法只是比 ref 访问方法多扫描了一些值为 NULL 的二级索引记录。

小贴士

值为 NULL 的记录会被放在索引的最左边。

10.5 range

在对索引列与某一个常数进行等值比较时，才会使用到前文介绍的几种访问方法（ref_or_null 比较奇特，还计算了值为 NULL 的情况）。但是有时我们面对的搜索条件很复杂，比如下面这个查询：

```
SELECT * FROM single_table WHERE key2 IN (1438, 6328) OR (key2 >= 38 AND key2 <= 79);
```

如果使用 idx_key2 执行该查询，那么对应的扫描区间就是 [1438, 1438]、[6328, 6328] 以及 [38, 79]。设计 MySQL 的大叔把“使用索引执行查询时，对应的扫描区间为若干个单点扫描区间或者范围扫描区间”的访问方法称为 range（仅包含一个单点扫描区间的访问方法不能称为 range 访问方法，扫描区间为 (− ∞ , + ∞) 的访问方法也不能称为 range 访问方法）。

10.6 index

来看下面这个查询：

```
SELECT key_part1, key_part2, key_part3 FROM single_table WHERE key_part2 = 'abc';
```

由于 key_part2 并不是联合索引 idx_key_part 的索引列中最左边的列，所以无法形成合适的范围区间来减少需要扫描的记录数量，从而无法使用 ref 或者 range 访问方法来执行这个语句。但是这个查询符合下面这两个条件：

- 它的查询列表只有 key_part1、key_part2 和 key_part3 这 3 个列，而索引 idx_key_part 又恰好包含这 3 个列；
- 搜索条件中只有 key_part2 列，这个列也包含在索引 idx_key_part 中。

也就是说，我们可以直接遍历 idx_key_part 索引的所有二级索引记录，针对获取到的每一条二级索引记录，都判断 key_part2 = 'abc' 条件是否成立。如果成立，就从中读取出 key_part1、key_part2、key_part3 这 3 个列的值并将它们发送给客户端。很显然，在这种使用 idx_key_part 索引执行上述查询的情况下，对应的扫描区间就是 (− ∞ , + ∞)。

由于二级索引记录比聚簇索记录小得多（聚簇索引记录要存储用户定义的所有列以及隐藏列，而二级索引记录只需要存放索引列和主键），而且这个过程也不用执行回表操作，所以直接扫描全部的二级索引记录比直接扫描全部的聚簇索引记录的成本要小很多。设计 MySQL 的大叔就把这种扫描全部二级索引记录的访问方法称为 index 访问方法。

另外，当通过全表扫描对使用 InnoDB 存储引擎的表执行查询时，如果添加了“ORDER BY 主

键”的语句，那么该语句在执行时也会被人为地认定为使用的是 index 访问方法，如下面这个查询：

```
SELECT * FROM single_table ORDER BY id;
```

10.7 all

最直接的查询执行方式就是全表扫描（我们已经提了无数遍了），对于 InnoDB 表来说也就是直接扫描全部的聚簇索引记录。设计 MySQL 的大叔把这种使用全表扫描执行查询的访问方法称为 all 访问方法。

10.8 注意事项

10.8.1 重温二级索引 + 回表

在使用索引来减少需要扫描的记录数量时，一般情况下只会为单个索引生成扫描区间。比如下面这个查询：

```
SELECT * FROM single_table WHERE key1 = 'abc' AND key2 > 1000;
```

查询优化器会识别到这个查询中的两个搜索条件：

- key1 = 'abc'；
- key2 > 1000。

如果使用 idx_key1 执行查询，对应的扫描区间就是 ['abc', 'abc']；如果使用 uk_key2 执行查询，对应的扫描区间就是（100, + ∞）。优化器会通过访问表中的少量数据或者直接根据事先生成的统计数据，来计算 ['abc','abc'] 扫描区间包含多少条记录，再计算（100,+ ∞）扫描区间包含多少条记录，之后再通过一定算法来计算使用这两个扫描区间执行查询时的成本分别是多少，最后选择成本更小的那个扫描区间对应的索引执行查询（有关选择使用哪个索引执行查询的具体步骤，会在第 12 章中详细唠叨）。

一般来说，等值查找比范围查找需要扫描的记录数更少（也就是 ref 访问方法一般比 range 访问方法好；但这并不总是成立，也有可能在采用 ref 方法访问时，相应的索引列为特定值的行数特别多）。我们假设优化器决定使用 idx_key1 索引来执行查询，那么整个查询的执行过程如下所示。

步骤 1. 先通过 idx_key1 对应的 B+ 树定位到扫描区间 ['abc', 'abc'] 中的第一条二级索引记录。

步骤 2. 根据从步骤 1 中得到的二级索引记录的主键值执行回表操作，得到完整的用户记录，再检测该记录是否满足 key2>1000 条件。如果满足则将其发送给客户端，否则将其忽略。

步骤 3. 再根据该记录所在的单向链表找到下一条二级索引记录，重复步骤 2 中的操作，直到某条二级索引记录不满足 key1 = 'abc' 条件为止。

小贴士

从上文可以看出，每次从二级索引中读取到一条记录后，就会根据该记录的主键值执行回表操作。而在某个扫描区间中的二级索引记录的主键值是无序的，也就是说这些二级索引记录对应的聚簇索引记录所在的页面的页号是无序的。每次执行回表操作时都相当于要随机读取一个聚簇索引页面，而这些随机 I/O 带来的性能开销比较大。于是设计 MySQL 的大叔提出了一个名为 Disk-Sweep Multi-Range Read（MRR，多范围读取）的优化措施，即先读取一部分二级索引记录，将它们的主键值排好序之后再统一执行回表操作。相对于每读取一条二级索引记录就立即执行回表操作，这样会节省一些 I/O 开销。当然使用这个 MRR 优化措施的条件比较苛刻，我们之前的讨论中没有涉及 MRR，之后的讨论中也将忽略这项优化措施，直接认为每读取一条二级索引记录就立即执行回表操作。大家如果对 MRR 感兴趣，可以查询官方文档。

10.8.2 索引合并

前文说过，MySQL 在“一般情况下”只会为单个索引生成扫描区间，但还存在特殊情况。在这些特殊情况下，MySQL 也可能为多个索引生成扫描区间。设计 MySQL 的大叔把这种使用多个索引来完成一次查询的执行方法称为 index merge（索引合并）。具体的索引合并方法有下面 3 种。

1. Intersection 索引合并

比如现在有下面这个查询：

```
SELECT * FROM single_table WHERE key1 = 'a' AND key3 = 'b';
```

我们当然可以选择使用全表扫描的方式执行该查询，不过由于搜索条件涉及 key1 和 key3 列，因此也可以使用下面两种方案执行该查询。

- 方案 1：使用 idx_key1 索引执行该查询，此时对应的扫描区间就是 ['a', 'a']。对于获取到的每条二级索引记录，根据它的 id 值执行回表操作后获取到完整的用户记录，再判断 key3= 'b' 条件是否成立。这里需要注意，扫描区间 ['a', 'a'] 是一个单点扫描区间，也就是说位于该区间中的二级索引记录，其 key1 列的值都为 'a'。这也就意味着这些二级索引记录其实是按照主键值进行排序的。
- 方案 2：使用 idx_key3 索引执行该查询，此时对应的扫描区间就是 ['b', 'b']。对于获取到的每条二级索引记录，根据它的 id 值执行回表操作后获取到完整的用户记录，再判断 key1= 'a' 条件是否成立。这里需要注意，扫描区间 ['b', 'b'] 是一个单点扫描区间，也就是说位于该区间中的二级索引记录，其 key3 列的值都为 'b'。这也就意味着这些二级索引记录其实是按照主键值进行排序的。

其实除了全表扫描以及上面提到的方案 1 和方案 2 之外，还可以有方案 3，具体如下。

- 方案 3：同时使用 idx_key1 和 idx_key3 执行查询。也就是在 idx_key1 中扫描 key1 值在 ['a', 'a'] 区间中的二级索引记录，同时在 idx_key3 中扫描 key3 值在 ['b', 'b'] 区间中的二级索引记录，然后从两者的操作结果中找出 id 列值相同的记录（即找出它们共有的 id 值）。然后再根据这些共有的 id 值执行回表操作（那些仅在单个扫描区间中包含的 id 值就不需要执行回表操作了），这样可能省下很多回表操作带来的开销。

这里的方案3就是所谓的Intersection索引合并。Intersection的中文含义就是“交集”，Intersection索引合并指的就是对从不同索引中扫描到的记录的id值取交集，只为这些id值执行回表操作。如果使用Intersection索引合并的方式执行查询，并且每个使用到的索引都是二级索引的话，则要求从每个索引中获取到的二级索引记录都是按照主键值排序的。比如在上面的查询中，在idx_key1的['a', 'a']扫描区间中的二级索引记录都是按照主键值排序的，在idx_key3的['b', 'b']扫描区间中的二级索引记录也都是按照主键值排序的。

为什么会要求从不同二级索引中获取到的二级索引记录都按照主键值排好序呢？这主要出于两方面的考虑：

- 从两个有序集合中取交集比从两个无序集合中取交集要容易得多；
- 如果获取到的id值是有序排列的，则在根据这些id值执行回表操作时就不再是进行单纯的随机I/O（这些id值是有序的），从而会提高效率。

假设idx_key1的扫描区间['a', 'a']中二级索引记录的id值是排好序的，且顺序为1、3、5；idx_key3的扫描区间['b', 'b']中二级索引记录的id值也是排好序的，且顺序为2、3、4，那么这个查询在使用Intersection索引合并来执行时，过程如下所示。

步骤1. 先从idx_key1索引的扫描区间['a', 'a']中取出第一条二级索引记录，该记录的主键值为1。然后从idx_key3索引的扫描区间['b', 'b']中取出第一条二级索引记录，该记录的主键值为2。因为1<2，所以直接把从idx_key1索引中取出的那条主键值为1二级索引记录丢弃。

步骤2. 接着继续从idx_key1索引的扫描区间['a', 'a']中取出下一条二级索引记录，该记录的主键值为3。步骤1中从idx_key3索引的扫描区间['b', 'b']中取出的二级索引记录的主键值为2。因为3>2，所以直接把步骤1中从idx_key3索引的扫描区间['b', 'b']中取出的主键值为2的那条二级索引记录丢弃。

步骤3. 接着继续从idx_key3索引的扫描区间['b', 'b']中取出下一条二级索引记录，该记录的主键值为3。步骤2中从idx_key1索引的扫描区间['a', 'a']中取出的二级索引记录的主键值为3。因为3=3，也就意味着获取主键交集成功，然后根据该主键值执行回表操作，获取到完整的用户记录后将其发送给客户端。

步骤4. 接着从idx_key1索引的扫描区间['a', 'a']中取出下一条二级索引记录，该记录的主键值为5。然后从idx_key3索引的扫描区间['b', 'b']中取出下一条二级索引记录，该记录的主键值为4。因为5>4，所以直接把从idx_key3索引的扫描区间['b', 'b']中取出的那条主键值为4的二级索引记录丢弃。

步骤5. 接着从idx_key3索引的扫描区间['b', 'b']中取出下一条符合条件的二级索引记录。发现没有了，然后结束查询。

别看这里写得啰嗦，其实这个执行过程可快了。

如果在使用某个二级索引执行查询时，从对应的扫描区间中读取出的二级索引记录不是按照主键值排序的，则不可以使用Intersection索引合并来执行查询。比如下面这个查询：

```
SELECT * FROM single_table WHERE key1 > 'a' AND key3 = 'b' ;
```

因为从idx_key1的扫描区间（'a', +∞）获取到的记录并不是按照主键值排序的，所以上

述查询不能使用 Intersection 索引合并的方式执行。

再看另一个例子：

```
SELECT * FROM single_table WHERE key1 = 'a' AND key_part1 = 'a';
```

对于 idx_key_part 索引来说，它的二级索引记录是先按照 key_part1 列的值进行排序的；在 key_part1 值相同的情况下，再按照 key_part2 值进行排序。那么在 idx_key_part 二级索引中，key_part1 值为 'a' 的二级索引记录并不是按照主键值进行排序的，所以上述查询也不能使用 Intersection 索引合并的方式执行。

另外，聚簇索引是比较特殊的存在，因为聚簇索引记录本身就是按照主键值进行排序的。比如对于下面这个查询：

```
SELECT * FROM single_table WHERE key1 = 'a' AND id > 9000;
```

从 idx_key1 的扫描区间 ['a', 'a'] 中获取的二级索引记录是按照主键值排序的，从聚簇索引的扫描区间（9000, + ∞）中获取的聚簇索引记录也是按照主键值排序的，所以上述查询可以使用 Intersection 索引合并的方式执行。不过实际在实现这种包含聚簇索引的 Intersection 索引合并方法时，并不会真正地扫描聚簇索引记录。那么它是怎么实现的呢？大家都知道，二级索引记录是包含索引列和主键列的，在索引列值相同的情况下，二级索引记录是按照主键值的大小排序的。所以上述查询在使用 Intersection 索引合并时，搜索条件 id > 9000 其实并不会为聚簇索引形成扫描区间 [9000, + ∞]，而是与搜索条件 key1 = 'a' 一起为 idx_key1 形成扫描区间 (('a', 9000), ('a', + ∞))。也就是说我们可以直接使用 idx_key1 执行查询，定位到符合 key1 = 'a' AND id > 9000 条件的第一条二级索引记录，然后沿着记录所在的单向链表向后扫描，直到某条记录不符合 key1 = 'a' 条件或者 id > 9000 条件为止。当然，针对获取到的每一条二级索引记录，都需要执行回表操作。在这个过程中不需要扫描聚簇索引的扫描区间（9000, + ∞）中的聚簇索引记录。

2. Union 索引合并

比如现在有下面这个查询：

```
SELECT * FROM single_table WHERE key1 = 'a' OR key3 = 'b'
```

我们能仅使用 idx_key1 或者 idx_key3 执行上述查询吗？不行！以 idx_key1 为例，假如使用 idx_key1 执行上述查询，那么对应的扫描区间就是（- ∞ , + ∞），而且需要针对获取到的每一条二级索引记录，都执行回表操作。在这种情况下是不使用 idx_key1 执行该查询的。

那么，我们就只能使用全表扫描的方式执行上述查询了吗？也不是。我们可以同时使用 idx_key1 和 idx_key3 来执行查询。也就是在 idx_key1 中扫描 key1 值位于 ['a', 'a'] 区间中的二级索引记录，同时在 idx_key3 中扫描 key3 值位于 ['b', 'b'] 区间中的二级索引记录，然后根据二级索引记录的 id 值在两者的结果中进行去重，再根据去重后的 id 值执行回表操作，这样重复的 id 值只需回表一次。这种方案就是所谓的 Union 索引合并。Union 的中文含义就是“并集”，Union 索引合并指的就是对从不同索引中扫描到的记录的 id 值取并集，为这些 id 值执行回表操作。

如果使用 Union 索引合并的方式执行查询，并且每个使用到的索引都是二级索引的话，则

要求从每个索引中获取到的二级索引记录都是按照主键值排序的。比如在上面的查询中，在 idx_key1 的 ['a', 'a'] 扫描区间中的二级索引记录都是按照主键值排序的，在 idx_key3 的 ['b', 'b'] 扫描区间中的二级索引记录也都是按照主键值排序的。这也是出于下面两个方面的考虑：

- 从两个有序集合执行去重操作比从两个无序集合中执行去重操作容易一些；
- 如果获取到的 id 值是有序的话，那么在根据这些 id 值执行回表操作时就不是进行单纯的随机 I/O（这些 id 值是有序的），从而会提高效率。

如果在使用某个二级索引执行查询时，从对应的扫描区间中读取出的二级索引记录不是按照主键值排序的，则不可以使用 Union 索引合并的方式执行查询。比如下面这个查询：

```
SELECT * FROM single_table WHERE key1 > 'a' OR key3 = 'b' ;
```

因为从 idx_key1 的扫描区间（'a', + ∞）中获取到的记录并不是按照主键值排序的，所以上述查询不能使用 Union 索引合并的方式执行。

再看另一个例子：

```
SELECT * FROM single_table WHERE key1 = 'a' AND key_part1 = 'a';
```

对于 idx_key_part 索引来说，它的二级索引记录先按照 key_part1 列的值进行排序，在 key_part1 值相同的情况下，再按照 key_part2 值进行排序。那么在 idx_key_part 二级索引中，key_part1 值为 'a' 的二级索引记录并不是按照主键值进行排序的，所以上述查询也不能使用 Union 索引合并的方式执行。

另外，聚簇索引是比较特殊的存在，因为聚簇索引记录本身就是按照主键值进行排序的。比如对于下面这个查询：

```
SELECT * FROM single_table WHERE key1 = 'a' OR id > 9000;
```

从 idx_key1 的扫描区间 ['a', 'a'] 中获取的二级索引记录是按照主键值排序的，从聚簇索引的扫描区间（9000, + ∞）中获取的聚簇索引记录也是按照主键值排序的，所以上述查询也可以使用 Intersection 索引合并的方式执行。

对于下面这个查询：

```
SELECT * FROM single_table WHERE (key_part1 = 'a' AND key_part2 = 'b' AND key_part3 = 'c') OR
(key1 = 'a' AND key3 = 'b');
```

我们可以先通过 idx_key1 和 idx_key3 执行 Intersection 索引合并，这样可以找到与搜索条件（key1 = 'a' AND key3 = 'b'）匹配的记录，然后再通过 idx_key_part 执行 Union 索引合并即可。

3. Sort-Union 索引合并

Union 索引合并的使用条件太苛刻，它必须保证从各个索引中扫描到的记录的主键值是有序的。比如下面这个查询就无法使用 Union 索引合并：

```
SELECT * FROM single_table WHERE key1 < 'a' OR key3 > 'z'
```

不过 key1<'a' 和 key3>'z' 这两个条件又特别让我们动心，所以我们可以这样操作：

- 先根据 key1<'a' 条件从 idx_key1 二级索引中获取二级索引记录，并将获取到的二级索

引记录的主键值进行排序；

- 再根据 key3 > 'z' 条件从 idx_key3 二级索引中获取二级索引记录，并将获取到的二级索引记录的主键值进行排序；
- 因为上述两个二级索引主键值都是排好序的，所以剩下的操作就与 Union 索引合并方式一样了。

我们把上面这种“先将从各个索引中扫描到的记录的主键值进行排序，再按照执行 Union 索引合并的方式执行查询”的方式称为 Sort-Union 索引合并。很显然，Sort-Union 索引合并比单纯的 Union 合并多了一步对二级索引记录的主键值进行排序的过程。

小贴士

为啥有 Sort-Union 索引合并，而没有 Sort-Intersection 索引合并呢？是的，在 MySQL 中确实没有 Sort-Intersection 索引合并这一说，不过在 MySQL 的近亲——MariaDB 数据库中实现了 Sort-Intersection 索引合并。

按照我的理解，Sort-Union 索引合并针对的是“单独根据搜索条件从某个二级索引中获取的记录数比较少”的使用场景，这样即使对这些二级索引记录按照主键值进行排序，成本也不会太高。而 Intersection 索引合并针对的是“单独根据搜索条件从某个二级索引中获取的记录数太多，导致回表成本太大”的使用场景，使用 Intersactions 索引合并后可以明显降低回表成本。但是，如果加入 Sort-Intersection 索引合并，就需要为大量的二级索引记录按照主键值进行排序，这个成本可能比使用单个二级索引执行查询的成本都要高，于是设计 MySQL 的大叔也就没有引入 Sort-Intersection 这个玩意儿。

10.9 总结

查询语句在本质上是一种声明式的语法，具体执行方式有很多种。设计 MySQL 的大叔根据不同的场景划分了很多种访问方法，比如：

- const；
- ref；
- ref_or_null；
- range；
- index；
- all；
- index_merge。

有的查询可以使用索引合并的方式利用多个索引完成查询，具体方法有下面 3 种：

- Intersection 索引合并；
- Union 索引合并；
- Sort-Union 索引合并。